FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA

ANA FLAVIA CAMELO, RM561489, TURMA 2TDSPV
GUSTAVO KENJI TERADA, RM562745, TURMA 2TDSPV
JOÃO GUILHERME CARVALHO NOVAES, RM566234, TURMA 2TDSPV
LUCAS FIGUEIREDO VIEIRA, RM561342, TURMA 2TDSPV
PEDRO CHASCI PUGA, RM565154, TURMA 2TDSPV

EMPRESA VITALIS

VITALIS

PROJETO PETHUB

São Paulo

2026

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA

ANA FLAVIA CAMELO, RM561489, TURMA 2TDSPV
GUSTAVO KENJI TERADA, RM562745, TURMA 2TDSPV
JOÃO GUILHERME CARVALHO NOVAES, RM566234, TURMA 2TDSPV
LUCAS FIGUEIREDO VIEIRA, RM561342, TURMA 2TDSPV
PEDRO CHASCI PUGA, RM565154, TURMA 2TDSPV

PROJETO PETHUB

Sprint 4 apresentado à Faculdade de Informática e Administração Paulista como requisito de nota para a avaliação da disciplina DISRUPTIVE ARCHITECTURES IOT, IOB & GENERATIVE AI.

São Paulo

2026

SUMÁRIO

1. Problemática **4**
2. Solução **5**
2.1 Abordagem de IA Adotada e Justificativa Técnica 5
2.2 Contribuição da IA para a Solução 6
2.3 Jornada do Usuário 6
3. Arquitetura da Solução **8**
3.1 Fluxo de Dados: Geração de Documentos por Consulta 9
3.1.1 Dados Utilizados pela IA Para a Geração de Documentos 10
3.2 Fluxo de Dados: Chat Diagnóstico com n8n 10
3.2.1 Dados Utilizados pela IA Para o Chat Diagnóstico 11
3.3 Fluxo de dados do backend do responsável (.NET) 11
4. Implementação da IA **12**
4.1 Prompt e saída estruturada 12
4.2 Validação e tratamento de erros 12
4.3 Segurança e privacidade 12
4.4 Resumo de exames 13
5. Testes e validação **13**
6. Resultados e limitações **13**
7. Repositórios e tecnologias **13**
7.1 Repositórios 13
7.2 Tecnologias: 13
8. Vídeo da entrega **13**

1. Problemática

O mercado pet brasileiro tem uma falha estrutural: o responsável procura a clínica quase só em urgências ou em gatilhos pontuais, como a vacinação. Por isso as consultas preventivas são raras, cada animal rende menos para a clínica ao longo da vida e não existe um histórico clínico contínuo entre clínica e responsável.

As informações do pet também ficam espalhadas, em prontuários de papel e em registros de clínicas diferentes, e ninguém enxerga a saúde do animal como um todo.

2. Solução

O PetHub é uma plataforma digital de cuidado veterinário contínuo. Ela liga responsáveis, veterinários e clínicas, do agendamento da consulta ao prontuário gerado com apoio de IA.
O diferencial está na IA dentro do atendimento real. Durante a consulta, o veterinário fala e o aplicativo transcreve a fala no próprio aparelho. O texto vai para o backend, e um modelo de linguagem devolve um rascunho com anamnese, receita (medicamentos e exames) e uma sugestão para avaliação do veterinário. Ele revisa, corrige e aprova. Nada é gravado antes dessa aprovação, então o preenchimento manual diminui e a decisão clínica continua com o profissional.
Há ainda um chatbot exclusivo para cada pet, voltado ao veterinário. Ele consulta o histórico do paciente e responde com esse contexto, diferente de um chatbot genérico.
A plataforma opera sobre três pilares:

- Preventivo: controle de vacinas, tratamentos e consultas. Quando o usuário permite, o aplicativo envia notificações no celular sobre as doses de medicamento e sobre consultas próximas, com aviso uma hora antes. A confirmação de cada consulta agendada também chega por e-mail.
- Terapêutico: histórico clínico concentrado (consultas, exames, diagnósticos, pedidos médicos e vacinas), disponível para responsável e veterinário a qualquer momento.
- IA generativa na consulta: o veterinário fala, o aplicativo transcreve e o modelo gera o rascunho dos documentos para revisão, como descrito acima.

2.1 Abordagem de IA Adotada e Justificativa Técnica

A abordagem é IA generativa com um LLM (Large Language Model). Usamos o Groq, plataforma que hospeda modelos de código aberto, com o modelo gpt-oss-120b.
Consideramos e descartamos outras abordagens. Um modelo preditivo treinado exigiria um conjunto grande de dados clínicos veterinários rotulados, e os conjuntos públicos disponíveis são pequenos, o que inviabiliza essa rota para um MVP. Um sistema de regras não acompanha a variação do discurso médico em texto livre. O NLP clássico identifica entidades isoladas, mas só redige documentos estruturados com um pipeline próprio extenso. Um LLM entende o português médico falado, estrutura os documentos e dispensa dados de treinamento nossos.
O planejamento da entrega anterior previa o Google Gemini. A implementação usa o Groq, cuja API segue o formato da OpenAI. Isso permitiu integrar o backend Java por um cliente HTTP comum e o n8n por um nó nativo.
Como o modelo é de código aberto, ele pode ser trocado por modelos mais enxutos. Se o veterinário quiser cortar custos, esses modelos menores podem rodar na

própria máquina. Isso é viável porque deixamos a saída bem fechada: o modelo só responde em JSON, num formato definido. Assim, a tarefa dele se resume ao básico, extrair informações de uma transcrição, e isso não exige um modelo muito potente.

2.2 Contribuição da IA para a Solução

A IA contribui em quatro frentes.
Na personalização, os documentos nascem do que foi dito naquela consulta, e a receita sai com os dados do pet, do responsável, do veterinário e da unidade já cadastrados.
No apoio à decisão, o modelo propõe sugestões de acompanhamento coerentes com a consulta, como exames complementares, retorno e cuidados em casa. O texto sempre começa com "Sugestão para avaliação do veterinário:" e não afirma diagnóstico definitivo.
Na recomendação de serviços, o modelo identifica os medicamentos e exames que o veterinário pediu de forma explícita e os coloca na receita. Quando a receita informa quantas vezes ao dia e por quantos dias, o serviço do responsável gera as doses, e o aplicativo agenda uma notificação para cada uma.
No chat com IA, o veterinário discute possíveis soluções para o caso e consulta os exames do pet de forma rápida, direto pela conversa. O chat responde com base no histórico do animal em questão.

2.3 Jornada do Usuário

O diagrama a seguir mostra as interações do responsável e do veterinário ao longo do cuidado.
A jornada começa no agendamento pelo responsável, que recebe a confirmação por e-mail. No atendimento, a IA trabalha nos bastidores: o aplicativo transcreve a fala e o sistema gera o rascunho dos documentos para o veterinário revisar. Depois da aprovação, o responsável baixa a receita em PDF e, se tiver permitido, recebe no celular as notificações das doses e das consultas próximas. É isso que mantém o responsável ligado à saúde do pet entre uma consulta e outra.



Figura 1 — Diagrama de jornada do usuário.

3. Arquitetura da Solução

A solução tem quatro partes: o aplicativo, dois serviços de backend, o n8n e a infraestrutura de dados e mensagens.
O aplicativo móvel é feito em React Native com Expo e atende os dois perfis, responsável e veterinário, cada um com sua navegação. É nele que a voz da consulta é transcrita (com o reconhecimento de voz do aparelho), que o PDF da receita é gerado e que as notificações locais são agendadas.
O serviço clínico, em Java com Spring Boot, cuida de veterinários, pets, consultas, prontuários, exames, e-mail de confirmação e da integração com o modelo de linguagem. O veterinário faz login direto nele. O serviço do responsável é feito em .NET: cuida do cadastro e do login do responsável, de endereços, contatos e lembretes, e funciona como porta de entrada do responsável para o serviço clínico. O n8n hospeda o chat com IA.
Os dados ficam no Oracle da FIAP, onde cada serviço mantém as próprias tabelas. O MongoDB guarda apenas o histórico das mensagens da fila. Os lembretes trafegam por uma fila RabbitMQ: o serviço clínico publica e o Vitalis consome. Exames em PDF e fotos de pets ficam num bucket do Google Cloud Storage. O modelo de linguagem é acessado pelo Groq.

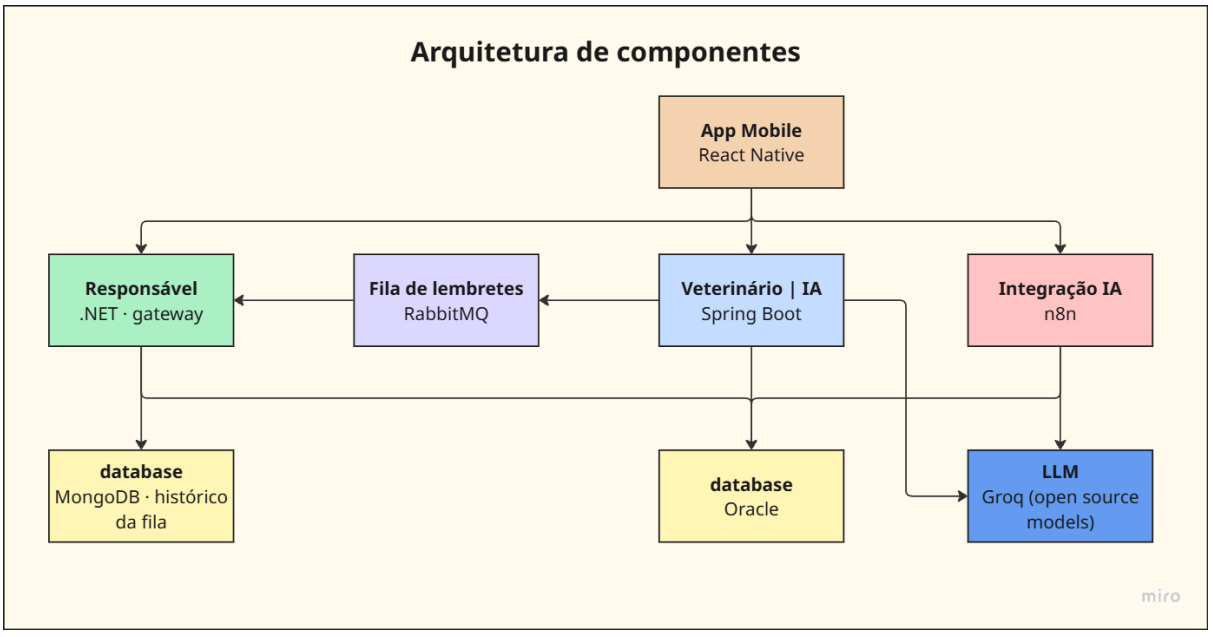


Figura 2 — Diagrama de arquitetura de componentes

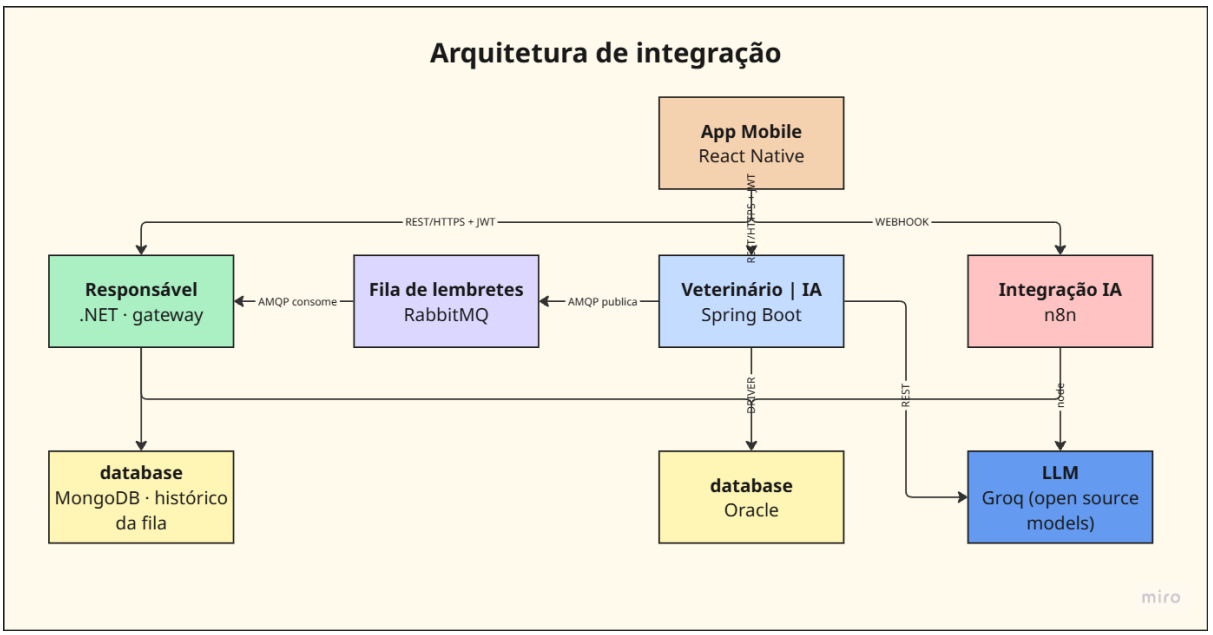


Figura 3 – Diagrama de arquitetura de integração

3.1 Fluxo de Dados: Geração de Documentos por Consulta

O fluxo começa no aplicativo, quando o veterinário inicia a consulta presencial. O aplicativo transcreve a fala no próprio aparelho e envia ao servidor apenas o texto. Nenhum áudio é enviado.

O texto segue para a IA, que devolve um rascunho com a anamnese, os medicamentos, os exames e a sugestão para avaliação. O sistema confere se a resposta veio completa antes de mostrá-la ao veterinário.

O veterinário revisa e edita o rascunho. Ao aprovar, o resultado é gravado no prontuário do pet e a consulta é marcada como realizada. Só então o responsável passa a ver a receita.

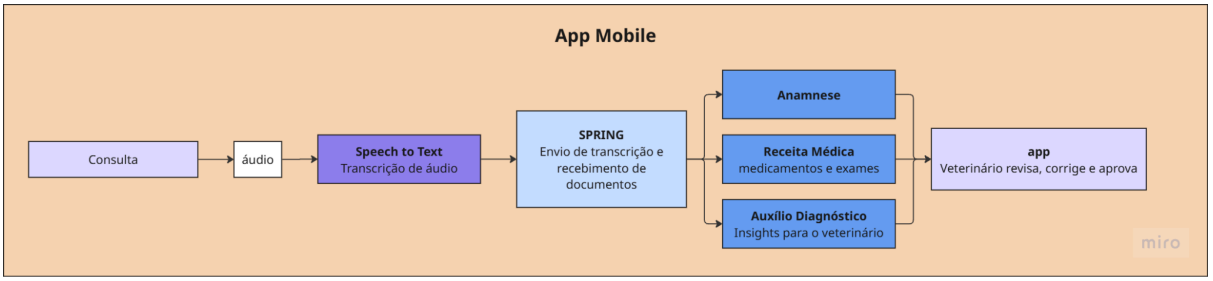


Figura 4 – Diagrama de fluxo de dados do App Mobile.

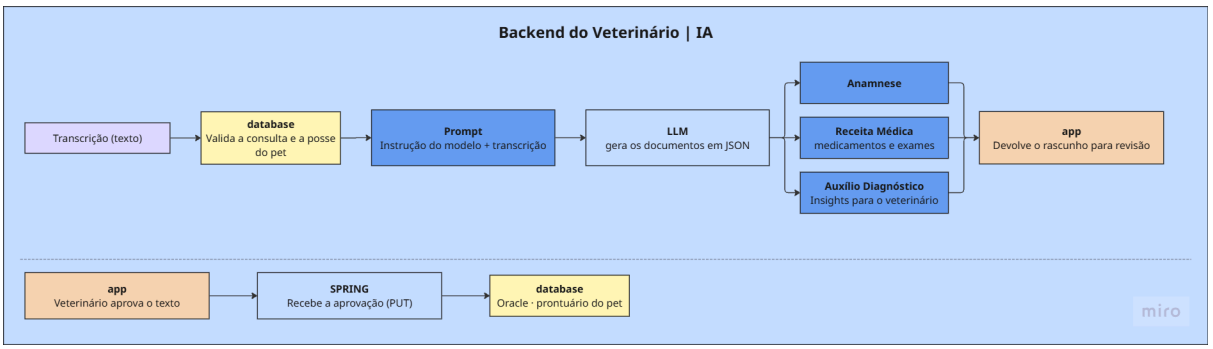


Figura 5 – Diagrama de fluxo de dados do backend do Veterinário.

3.1.1 Dados Utilizados pela IA Para a Geração de Documentos

O modelo que gera os documentos recebe duas coisas: a instrução fixa, que define o papel, as regras e o formato da resposta, e a transcrição da consulta, que é o texto livre.

Os dados do pet (nome, espécie, raça, nascimento e peso), do responsável, do veterinário e da unidade não passam por esse modelo. O sistema os usa apenas para preencher o cabeçalho da receita. Os dados do pet e do prontuário chegam à IA somente no chat, descrito na seção 3.2.

3.2 Fluxo de Dados: Chat Diagnóstico com n8n

O chat opera de forma independente do fluxo de consulta. O veterinário escreve no chatbot do aplicativo, e a mensagem chega ao n8n junto com o identificador do pet e o da sessão.

Dentro do n8n, o agente busca o prontuário do pet no Oracle, monta o prompt com a mensagem e esses dados e o envia ao Groq. A resposta, com insights diagnósticos, volta ao veterinário pelo chatbot. O n8n guarda a memória da conversa por sessão, e todo o processamento ocorre nele, sem passar pelo backend Java.

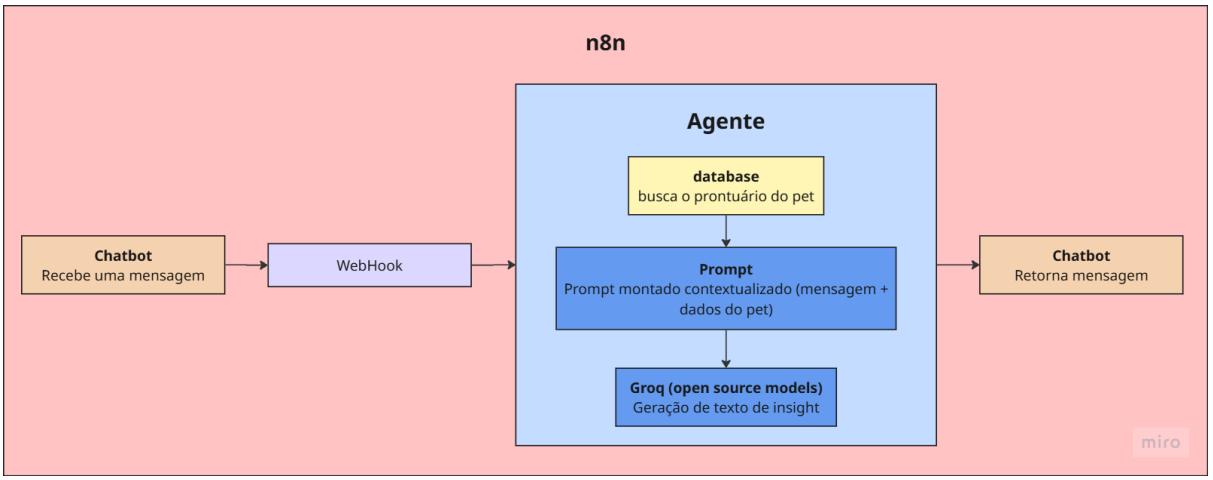


Figura 6 – Diagrama de fluxo de dados da integração n8n

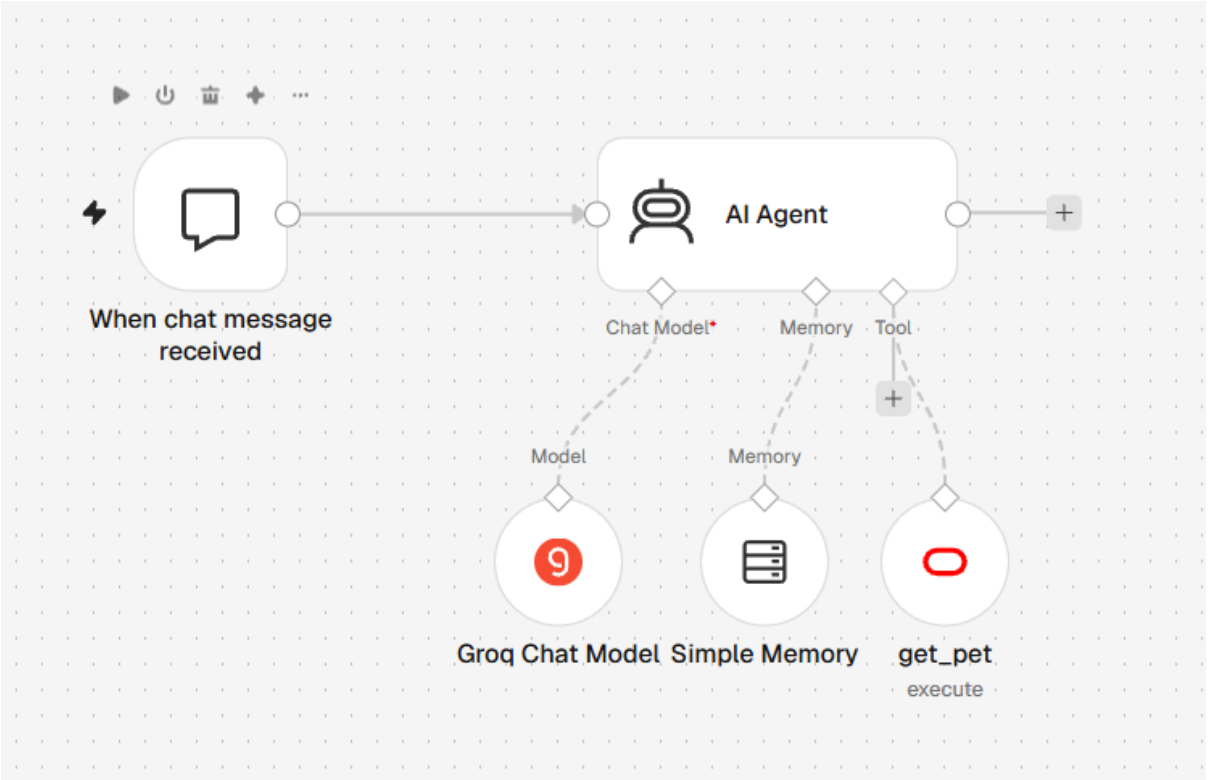


Figura 7 – Fluxo do n8n.

3.2.1 Dados Utilizados pela IA Para o Chat Diagnóstico

Para que as perguntas se apoiem no histórico real do animal, o agente usa o prontuário do pet no Oracle (consultas anteriores, diagnósticos, vacinas e exames) e a mensagem do veterinário. A memória da sessão permite perguntas de acompanhamento sem repetir o contexto.

3.3 Fluxo de dados do backend do responsável (.NET)

O .NET tem dois fluxos. No primeiro, o gateway clínico, o responsável usa pets, consultas e exames pelo aplicativo. O serviço valida o token do responsável, emite um token de serviço de 60 segundos e repassa a chamada ao serviço clínico, que aplica as regras de posse e acessa o Oracle. A resposta volta pelo .NET até o aplicativo.
No segundo, os lembretes, o serviço clínico publica uma mensagem no RabbitMQ depois de gravar a consulta. O consumidor do Vitalis processa a mensagem, grava o lembrete no Oracle e registra o histórico da fila no MongoDB. Quando a receita informa vezes ao dia e duração, um job do Vitalis libera as doses de hora em hora.O aplicativo lista os lembretes e agenda a notificação local de cada dose.

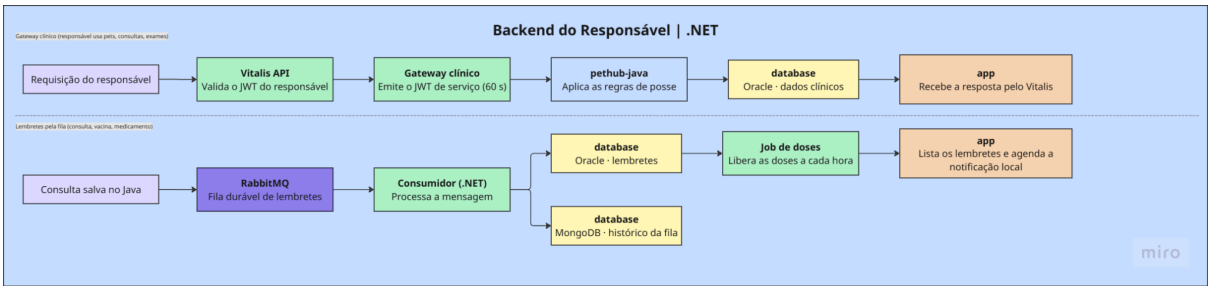


Figura 8 – Diagrama de fluxo de dados do backend do responsável.

4. Implementação da IA

4.1 Prompt e saída estruturada

A instrução do modelo fica em arquivos de texto no repositório, e ajustá-la não exige mexer em código. A instrução da consulta manda o modelo usar somente o que está na transcrição, sem inventar sintomas, doses, medicamentos nem exames. Quando uma informação não foi dita, a anamnese registra "Não informado na consulta.".

A resposta é um JSON com quatro campos: anamnese (texto), medicamentos (lista com nome, quantidade, posologia, observações, vezes por dia e duração em dias), exames (lista com nome e observações) e sugestões (texto). Às vezes por dia e a duração são números inteiros, ou nulos quando o veterinário não disse. Esses dois campos alimentam a geração das doses no Vitalis.

A transcrição entra na mensagem dentro de uma tag, e a instrução diz ao modelo que ela é material a processar, nunca ordem a obedecer. Se a transcrição trouxer a própria tag de fechamento, o sistema a remove antes do envio, para que o texto não consiga encerrar o bloco.

4.2 Validação e tratamento de erros

O backend recusa a análise quando faltam dados ou algo está errado: transcrição ausente (400), consulta inexistente (404), consulta cancelada (409). Se a IA estiver fora do ar, sem chave configurada ou devolver uma resposta incompleta, a API responde 503. Antes de devolver o rascunho, o Java limpa as listas de medicamentos e exames.

4.3 Segurança e privacidade

Só o veterinário gera e aprova a análise. O responsável apenas lê a receita já aprovada e não acessa dados de outro responsável. A chave do Groq fica no servidor e nunca vai ao aplicativo. O áudio não sai do aparelho, e nenhum documento é gravado sem aprovação.

4.4 Resumo de exames

O aplicativo converte o PDF de um exame em texto, e o serviço clínico envia esse texto ao Groq com uma instrução própria. O modelo devolve um resumo em JSON.

5. Testes e validação

Os testes e a demonstração do funcionamento da IA foram feitos e estão registrados no vídeo da entrega.

6. Resultados e limitações

O fluxo completo funcionou: o veterinário dita a consulta, revisa o rascunho e aprova, e o responsável recebe a receita e as notificações. Para o veterinário, o ganho é menos tempo preenchendo documentos. Para o responsável, é o acesso rápido à receita e às doses. Para a clínica, é um prontuário mais completo e um canal para trazer o responsável de volta antes da próxima urgência.
A principal limitação é depender de um sistema de IA de terceiros, que pode não estar disponível. A forma de contornar isso é usar um modelo local, instalado na máquina do veterinário. Essa alternativa tem uma limitação própria: o desempenho depende do equipamento de cada profissional e pode variar de uma máquina para outra.

7. Repositórios e tecnologias

7.1 Repositórios

Aplicativo móvel (React Native e Expo): https://github.com/FIAP-MOBILE-2026-Fevereiro/2tdspv-challenge-vitalis
Serviço clínico (Java e Spring Boot): https://github.com/vitalis-sa/pethub-java
Serviço do responsável (.NET): https://github.com/vitalis-sa/PetHub-.NET
Documentação da IA: https://github.com/vitalis-sa/pethub-ai

7.2 Tecnologias:

React Native com Expo, TypeScript, Java com Spring Boot, .NET 10, Oracle, MongoDB, RabbitMQ (CloudAMQP), n8n, Groq, Brevo e Google Cloud Storage.

8. Vídeo da entrega

Link do vídeo no YouTube: [colar o link aqui]